Arctic: Remote Attestation Server for Hardware-based Firmware Security

System Security

Introduction

Nowadays, many systems are shipped with TPM (Trusted Platform Module) chips onboard, either as a discrete security chip (also known as dTPM: Discrete TPM) or embedded inside the CPU dies and microcode (also known as fTPM: Firmware-based TPM). In addition, current versions of widely-used operating systems (like Windows 11) requires a TPM to be present on the system to allow installation or the upgrade of the operating system itself. Linux-based systems also offer kernel-controlled tight security structures through features like SELinux and most importantly, Linux IMA (Integrity Measurement Architecture) offers a completely verified execution environment for programs and users.

While operating system level security is mostly elaborated on, generally the security of the components until the OS/Bootloader execution doesn't get the same attention as the OS-level practices. Since these components might have obscure implementations (like the Firmware of a device, which is rarely open sourced), ensuring the integrity of them should be considered also important.

This project focuses on using specific features and characteristics delivered by the TPM to create and implement a completely end-to-end verified system security chain for platforms like desktops, laptops or servers. It utilizes the values of "Platform Configuration Registers" or simply PCRs and a remote server storing these values to detect any changes into the underlying components of the system against a well-known trusted state. This state will be referred to as "Golden State" or "Reference State".

Components

This project consists of 3 main elements, and different sub-elements:

1) Attestation Server
   - Node.js (with Express framework) for the API server and the attestation logic
   - MySQL DB for storing the measurements and related information.

2) OS-level Agent, collecting PCR values on-boot

3) User Interface, written in plain HTML+JS+CSS and is static.

For demonstration of this project, the Node.js server is deployed with Nginx inside a minimized image Ubuntu Server 24.04 VM. MySQL Database is deployed as an OCI container, and also additional VMs used basically as an "example host to be protected". They also run Ubuntu 26.04 (Development Branch) and Ubuntu 24.04 (LTS). All virtual machines are deployed inside a Proxmox Virtual Environment (Version 9.1.1) installation.

Platform Configuration Registers

Platform Configuration Registers (PCRs) are unique, volatile memory locations within a Trusted Platform Module, designed to store cryptographic hashes that represent the "state" of a computer. Unlike standard registers, PCRs cannot be overwritten with arbitrary data; they can only be updated through an "**extend**" operation. In this process, the TPM takes the current value of the PCR, appends the new data (usually a measurement of firmware, kernels, or drivers), and hashes the combination to create a new value:

$$\mathbf{PCR_{new} = hash(PCR_{old} \ || \ NewData)}$$

This "hash chain" ensures that the final value is cryptographically dependent on the exact sequence of everything that loaded before it, creating an immutable audit log of the boot process.

According to the Trusted Computing Group (TCG) guidelines, specific PCR indices are reserved for standardized measurements. For example, PCR 0 typically stores measurements of the Core Root of Trust for Measurement (CRTM) and BIOS/UEFI firmware, while PCR 7 is often used for Secure Boot state. By comparing these runtime values against "known good" measurements (this is called “attestation”, the result we are aiming for), the system can verify its integrity. If a single bit of any component, such as the Firmware (also known as BIOS/UEFI Image), Firmware settings (also known as BIOS settings), Option ROMs (OROMs) or the bootloader is altered by malware, the corresponding PCR value will change entirely, allowing the TPM to refuse to release encryption keys, and indicating different measurement results, effectively allowing the remote attestation server to clarify which component was modified.

Below is a table from Gentoo Linux Wiki, listing the PCRs and their corresponding components. Although PCR1 is noted as “BIOS Configuration”, in many occasions, it also includes changes to hardware such as RAM modules, CPUs and depending on the implementation, even the internal storage devices. Tests on both Virtual TPMs (developed by IBM) and physical Dell hosts (OptiPlex 7040M) confirm that hardware changes result in a different PCR1 value, even if the BIOS settings are preserved.

PCR	TCG Description	Notes
0	SRTM, BIOS, Host Platform Extensions, Embedded Option, ROMs and PI Drivers	Stores firmware, may change if the UEFI/BIOS is updated. Acts as the root for the chain of trust. Some implementations record CPU microcode measurements here.
1	Host Platform Configuration	The UEFI configuration itself, such as settings. This typically contains the entire contents of the CMOS/NVRAM, minus any dynamic or security-sensitive data.
2	UEFI driver and application Code	Option ROMs
3	UEFI driver and application Configuration and Data	Option ROM configuration
4	UEFI Boot Manager Code (usually the MBR) and Boot Attempts	Measures manager code itself, and that a use was attempted
5	Boot Manager Code Configuration and Data (for use by the Boot Manager Code) and GPT/Partition Table	Measures the configuration of the boot device, including the GPT partition table of the device
6	Host Platform Manufacturer Specific	May be used for S4 and S5 Power State Events
7	Secure Boot Policy	Measures contents of Secure Boot keys and certificates used to verify boot applications
8 - 15	Defined for use by the Static OS	
16	Debug	Optional, and sometimes unused PCR
23	Application Support	Can be set and reset by the OS

This project specifically focuses on checking PCR values 0 to 7 and individually comparing against a manually marked "golden state" to detect any modifications and integrity failures for the given components.

Implementation and Decisions

The Attestation Server consists of a Node.js Server, that can connect to a MySQL database. The MySQL database holds an inventory information about the hosts, also including customizable labels. The user is expected to manage their "fleet" through a Web Interface, which connects to the attestation server using the APIs provided by the server itself.

For the first installation, a simple Python script developed as an agent is installed on the target hosts to be protected. Then this script registers the server to the central Attestation Server, receiving a unique Host ID and a Host Authentication Password. The agents must supply these information, in addition to their measurements of the system state per each measurement submission, preventing possible attack vectors like a host trying to submit measurements on behalf of another one, ensuring measurement integrity. Please see Limitations section on how tamperproof measurements from the TPM to the server could be implemented, eliminating even the potential MitM attacks.

Agent runs only once per boot, since PCRs from PCR0 to PCR7, by their defined characteristics, are not expected to change after boot. In fact, for many systems, extending a value to these specific registers are prohibited by the operating systems. For custom measurement implementations using PCRs, this agent can simply be modified to report custom PCR values per a given time interval.

After receiving a measurement submission, the Attestation Server registers individual PCR values submitted by the agent to the database, and then compares them to the marked "golden state". If all the PCRs match, it registers the measurement as "Matched". If there are any mismatches, the measurement will be registered as a "Mismatch" and the host state will be updated as "Corrupt". The server also saves which specific PCR values had a mismatch, making it easy to understand which specific component was (probably in an unauthorized way) modified.

The user interface then allows the user to see the overall health status of the servers, by also allowing listing the specific measurement results and their details. This approach makes it very easy to detect potential mismatches, also keeping it easy to troubleshoot or detect any mismatch. It is also possible to set a measurement as a "Golden State / Reference Measurement" by simply clicking on a measurement and acknowledging the warning prompt.

For security and accountability reasons, the system is designed in a way that changing the Reference Measurement will not update the results of the previous measurements. This prevents making an unauthorized change and then setting a Reference Measurement to basically "erase" or counteract the previous mismatch results, which in a way aims to create a "tamper-resistant" backlog of measurements.

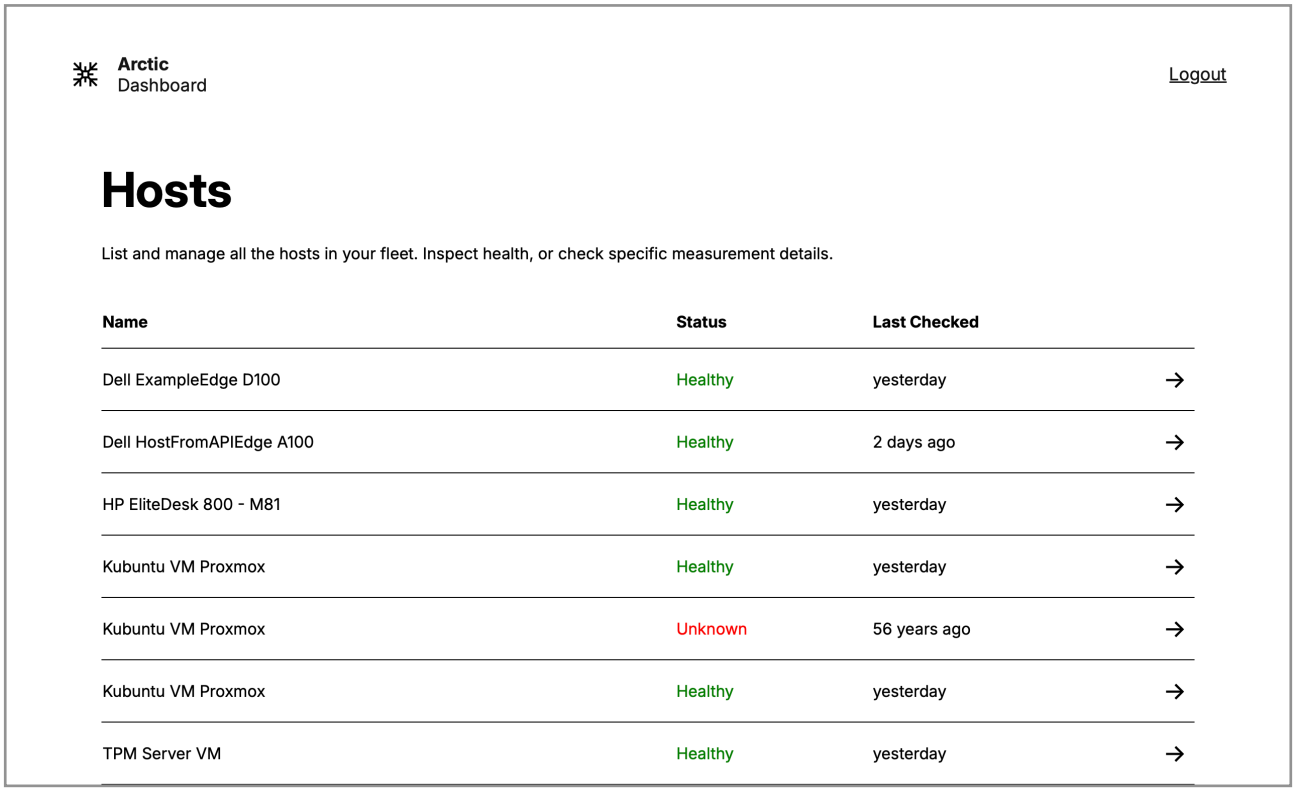
User Interface

The Dashboard lists all the hosts registered into an attestation server, alongside with their user-defined tags, the timestamp of the last measurement and current health status. There are 3 possible health status implemented:

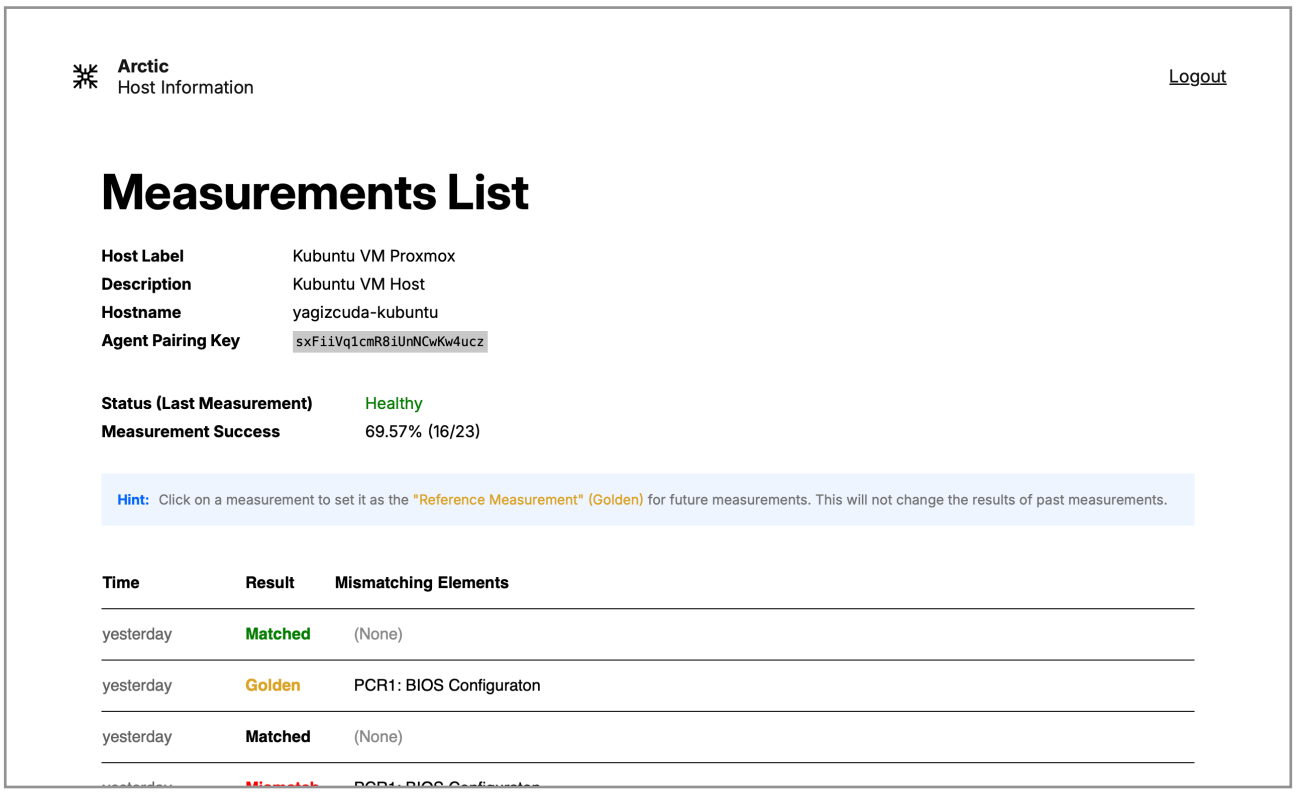
> **Healthy**: Indicates that the last measurement matched the Reference Measurement.
> **Corrupt**: Indicates that there is at least 1 mismatching PCR when compared with the Reference Measurement.
> **Unknown**: Indicates that the current status is unknown. This generally indicates a newly registered host without a Reference Measurement. To set a measurement as Reference, the steps are the same as changing the Reference Measurement.

Although Unknown does not mean a mismatch, it is still marked with Red in the user interface, since this state also means that it is not possible to check the integrity of the specific host, and must be fixed to ensure security.

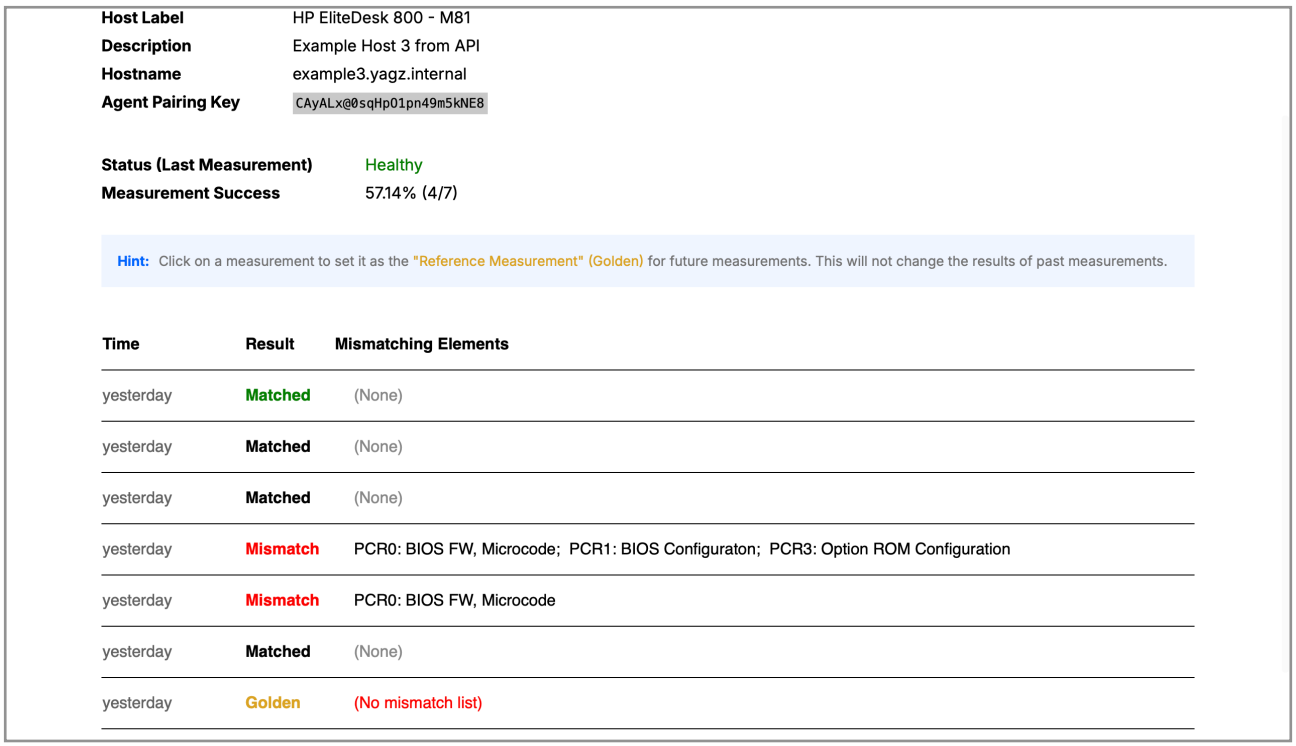
Below, some screenshots of the User Interface are provided.



The Hosts page of the of Arctic, listing the current states of the enrolled hosts (the fleet)



The Measurements List, also viewing the Overview of the specific host. Agent Pairing Key is referred



to as “Host Authentication Key” in this document.
Another example host, showing different examples of matched and mismatched measurements, alongside with the descriptions of mismatched PCRs.

Reference Measurement

In the context of Remote Attestation, a Reference Measurement is a pre-calculated, authoritative cryptographic hash that represents the expected "known good" state of a specific system component (like the Firmware, bootloader, or OS kernel). It is also called as “Golden Measurement” or “Known Good State”. It serves as the baseline against which the runtime PCR values of a remote client are compared. When the Remote Attestation Server receives a TPM quote from a client, it validates the integrity of the platform by ensuring the current measurements match these stored reference values. Any deviation indicates that the boot sequence or system configuration has been altered.

To set a Reference Measurement, first a measurement must be sent by the agent to the Attestation Server and it must be listed under the measurements list. As described in the previous sections, this is done to create a tamper-evident backlog of measurements. Then, the manager can simply click on a specific measurement and acknowledge the warning to set it as reference. Reference Measurements are indicated with gold-colored “Golden” keyword, indicating the current Reference Measurement.

This implementation focuses on having only one Reference Measurement to keep the project simple. Although it may be possible to use multiple Reference Measurements, then the logic used to calculate mismatching PCRs would needed to be evaluated and reworked on. For example, if PCR0 matches Reference Measurement 0 but PCR1 matches Reference Measurement 1, should the program accept this as a valid measurement, allowing mixing and matching, or should it try to match exact sequences from PCR0 to PCR7? In the latter case, if there are many mixes and matches, which Reference Measurement will be the actual reference for detecting mismatching PCR values?

Setting a new correctly Reference Measurement shows a success message. The User Interface then refreshes the list to indicate the new Reference Measurement.



A success message is displayed indicating that the Reference Measurement has been set/modified correctly.

Limitations

Mainly since I had limited time and wanted to keep this project simple, this project only implements detection of changing values, which is very powerful for many use cases. There are also other implementations making it possible to have a TPM-to-Server (or if a cloud server is used, can be also called as TPM-to-Cloud) attestation chain by verifying specific keys of TPM module (known as Attestation Keys and Endorsement Keys) to prevent possible modifications on the module itself, like possible supply chain attacks. In the mentioned implementation, the TPM sends a "quote", a signed message with PCR measurements, so it becomes impossible to modify the measurements by any MitM attack, either inside the Operating System or in the network.

References

This report has been created by devygz for the System Security exam's project on February 6, 2026.

All the tests and deployments have been performed on Ubuntu-based virtual machines running on Proxmox VE, using software TPM implementation (developed by IBM, default implementation of TPM inside Proxmox). Additional tests have been performed on a bare-metal hardware (Dell OptiPlex 7040M), equipped with a Discrete TPM chip, manufactured by Nuvoton (Manufacturer Code: NTCC).

This document uses Hoefler Text as the body, titles and headings font; Baskerville for numbers; and Helvetica Neue for Header & Footers.